

Universidade Federal de Mato Grosso (UFMT)

Faculdade de Engenharia

News Diff — Coleta e Análise de Notícias

Alunos: Pedro Reis, Gabriel Martins

Projeto e Análise de Algoritmos

Professor: Prof. Dr. Gustavo Post Sabin

Visão Geral

O **News Diff** é um sistema que coleta notícias de portais brasileiros via RSS, agrupa matérias similares (republicações) e gera um ranking dos assuntos mais repercutidos. O coração do sistema é um algoritmo de *LCS (Longest Common Subsequence)* que compara títulos de notícias para identificar similaridades, mesmo quando há pequenas variações textuais.

Ideia Principal

Jornais e portais frequentemente republicam matérias de agências ou uns dos outros, com pequenas alterações nos títulos. O News Diff:

- Coleta feeds RSS dos principais portais (G1, Folha, UOL, R7, Agência Brasil)
- Processa os títulos usando LCS para medir similaridade
- Agrupa notícias similares em "assuntos"
- Rankeia os assuntos mais republicados

Arquitetura do Sistema

O sistema é composto por quatro módulos principais, cada um com responsabilidades bem definidas:

- **main.py**: Interface interativa com o usuário (CLI com menu)
- **coletor.py**: Coleta RSS e parsing de feeds dos portais
- **lcs.py**: Implementação do algoritmo LCS com otimizações
- **analizador.py**: Agrupamento via LCS e geração de ranking

Estrutura do Projeto

```
1 news-diff/  
2     main.py           # Interface interativa (menu)  
3     coletor.py        # Coleta RSS e parsing de feeds  
4     analisador.py     # Agrupamento via LCS e ranking  
5     lcs.py            # Implementacao do algoritmo LCS  
6     portais_extra.txt # (opcional) Portais adicionais  
7     README.md         # Documentacao
```

Componentes Detalhados

1. Modelo de Dados (`coletor.py`)

A classe `Noticia` encapsula todas as informações de uma notícia:

- **titulo:** Título da notícia
- **resumo:** Descrição/lead da matéria (com HTML removido)
- **portal:** Nome do portal de origem
- **url:** Link para a notícia original
- **data:** Timestamp da publicação (com timezone)

Inclui métodos para serialização/desserialização JSON, permitindo snapshots offline.

2. Coletor RSS (`coletor.py`)

- Busca feeds RSS de portais pré-configurados (G1, Folha, UOL, R7, Agência Brasil)
- Suporta portais extras via arquivo `portais_extra.txt` (formato: `Nome | URL`)
- Parse robusto de datas com múltiplos formatos (`parse_data`)
- Limpeza de HTML dos resumos (`limpar_html`)
- Cache em JSON para modo offline (`salvar_json` / `carregar_json`)
- Delay entre requisições para não sobrecarregar servidores

3. Algoritmo LCS (`lcs.py`)

- Implementação clássica de programação dinâmica com tabela DP (`lcs_table`)
- Complexidade $O(m \times n)$ em tempo e espaço
- Função de backtrack para reconstrução da subsequência (útil para debug)
- Cache LRU (`@lru_cache`) no pré-processamento de textos
- Pré-processamento: remove pontuação, normaliza para minúsculas
- Similaridade normalizada: `lcs_len / max(len(A), len(B), 1)`

4. Analisador (analisador.py)

- Agrupa notícias similares usando LCS (limiar: 0.35)
- **Deduplicação por URL:** remove duplicatas antes do processamento
- **Filtro de templates editoriais:** descarta títulos como "VÍDEOS:" ou "assista aos telejornais"
- Pré-filtro por palavras-chave (mínimo 2 em comum) para evitar LCS desnecessário
- Cache de palavras-chave por notícia (evita reproprocessamento)
- **Representante fixo:** cada grupo usa a primeira notícia como representante permanente
- Gera estatísticas detalhadas de processamento

5. Interface Interativa (main.py)

- Menu interativo com 3 opções:
 1. Coletar notícias agora (modo online)
 2. Usar snapshot salvo (modo offline)
 3. Coletar e salvar snapshot (para uso futuro)
- Validação de entrada do usuário
- Cores no terminal para melhor experiência
- Configuração interativa de parâmetros (janela temporal, tamanho do ranking)

Como Instalar

Dependências do projeto:

```
1 feedparser >=6.0.12
2 newspaper3k >=0.2.8
3 requires-python = ">=3.13"
```

Usando uv (recomendado):

Windows:

```
1 powershell -ExecutionPolicy ByPass -c "irm
  https://astral.sh/uv/install.ps1 | iex"
```

Linux/macOS:

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh
```

Após instalar o uv:

```
1 uv sync
2 uv run main.py
```

Sem o uv, instale as dependências manualmente:

```
1 pip install feedparser newspaper3k
2 python main.py
```

Como Usar

Para executar o programa, basta rodar o comando abaixo no terminal:

```
1 uv run main.py
```

Ou, se não estiver usando uv:

```
1 python main.py
```

Ao iniciar, o programa apresenta um menu interativo:

```
NEWS DIFF

1. Coletar not cias agora
2. Usar snapshot salvo (offline)
3. Coletar e salvar snapshot

0. Sair

Escolha uma opção :
```

O usuário escolhe entre as opções digitando o número correspondente. Em seguida, o programa solicita os parâmetros necessários:

```
Escolha uma opção : 1
Janela de tempo em horas [padrão 5]:
Quantas not cias no ranking [padrão 10]:
```

Basta pressionar Enter para aceitar os valores padrão ou digitar um valor personalizado.

Portais Extras

Para adicionar portais personalizados, crie o arquivo `portais_extra.txt` no mesmo diretório:

```
1 # Formato: Nome do Portal | URL do RSS
2 Estadao | https://economia.estadao.com.br/feed
```

```
3 CNN Brasil | https://www.cnnbrasil.com.br/feed/  
4 IG | https://ultimosegundo.ig.com.br/rss/
```

Exemplo de Execução

Abaixo um exemplo real de execução do programa:

```
~/C/p/f/p/proj    uv run main.py  
  
NEWS DIFF  
  
1. Coletar not cias agora  
2. Usar snapshot salvo (offline)  
3. Coletar e salvar snapshot  
  
0. Sair  
  
Escolha uma op    o: 1  
Janela de tempo em horas [padr o 5]:  
Quantas not cias no ranking [padr o 10]:  
  
NEWS DIFF      Not cias do Momento  
  
Coletando not cias das    ltimas    5h...  
Per odo: 27/02 22:33      agora  
  
    G1... 0 not cias  
    Folha... 100 not cias  
    IG... 2 not cias  
  
Total coletado: 102 not cias  
  
Agrupando 102 not cias via LCS...  
    URLs duplicadas removidas: 3  
    Templates editoriais descartados: 5  
    Pr -filtro bloqueou 4.521 compara es  
    LCS executado 156 vezes  
    Total de compara es: 4.677
```

Análise de Complexidade

O algoritmo LCS possui complexidade $O(m \times n)$ em tempo e espaço, onde m e n são os tamanhos dos títulos comparados (em palavras). A alternativa de força bruta teria complexidade $O(2^n)$.

O pré-filtro por palavras-chave reduz dramaticamente as execuções do LCS. Em uma execução típica com 470 notícias em janela de 5 horas:

Métrica	Valor
Comparações totais	83.000
Bloqueadas pelo pré-filtro	82.700 (99,6%)
LCS efetivamente executado	300 (0,4%)

Tabela 1: Eficiência do pré-filtro por palavras-chave

A escolha do LCS sobre métricas como Jaccard ou distância de edição se justifica porque o LCS **preserva a ordem das palavras**, capturando similaridade estrutural entre títulos — não apenas sobreposição de vocabulário.

Detalhes Técnicos Relevantes

Algoritmo de Similaridade

- Pré-processamento: Remove pontuação, converte para minúsculas
- LCS: Encontra a maior subsequência comum entre dois textos
- Normalização: $\text{similaridade} = \text{tamanho_LCS} / \max(\text{tamanho_texto1}, \text{tamanho_texto2})$
- Limiar: 0.35 (configurável) — acima disso são considerados similares

Otimizações Implementadas

- **Cache de palavras-chave:** Evita reprocessar textos (dicionário por ID)
- **Pré-filtro por interseção:** Só executa LCS se houver ≥ 2 palavras em comum
- **LRU Cache no LCS:** Resultados de pré-processamento em cache (128 entradas)
- **Deduplicação por URL:** Notícias com a mesma URL são removidas antes do agrupamento
- **Filtro editorial:** Títulos que seguem padrões de template (ex: "VÍDEOS:", "assista aos telejornais") são descartados
- **Representante fixo:** Cada grupo utiliza sua primeira notícia como representante permanente, evitando instabilidade no cluster